

CatalogDemoData.xml

BreakDown MODULE-4 (PART 3)

1. Catalog Creation

XML:

```
<ProdCatalog  
  prodCatalogId="XML_SHOES_CATALOG"  
  catalogName="XML Shoes Catalog"/>
```

From the first image:

You can see entity:

ProdCatalog

Important columns:

prodCatalogId
catalogName

Record inserted:

prodCatalogId	catalogName
XML_SHOES_CATALOG	XML Shoes Catalog

Database equivalent:

```
INSERT INTO PROD_CATALOG
VALUES(
'XML_SHOES_CATALOG',
'XML Shoes Catalog'
);
```

Purpose:

This creates the top-level catalog.

Imagine Amazon:

Amazon
└── Shoes Catalog

2. Category Creation

XML:

```
<ProductCategory  
productCategoryId="XML_SHOES_ROOT"  
categoryName="Browse Root (XML)"/>
```

Several categories:

ROOT

MEN

MSPORTS

PROMO

From model:

Entity:

ProductCategory

Inserted records:

Category ID	Name
XML_SHOES_ROOT	Browse Root
XML_SHOES_MEN	Men

XML_SHOES	MSpor
_MSPORTS	ts
XML_PROMO	Promo
_CAT	tion

3. Category hierarchy (tree structure)

XML:

```
<ProductCategoryRollup  
productCategoryId="XML_SHOES_MEN"  
parentProductCategoryId="XML_SHOES_ROOT"/>
```

From diagram:

Entity:

ProductCategoryRollup

This creates parent-child relationships.

Tree becomes:

```
ROOT  
|  
├── MEN  
|   |
```



Without rollup:

ROOT

MEN

MSPORTS

PROMOTION

(all disconnected)

4. Linking Catalog to Categories

XML:

```
<ProdCatalogCategory  
  prodCatalogId="XML_SHOES_CATALOG"  
  productCategoryId="XML_SHOES_ROOT"  
  prodCatalogCategoryId="PCCT_BROWSE_ROOT"/>
```

Data model:

ProdCatalogCategory

This connects:

Catalog



Categories

Relationship:

XML_SHOES_CATALOG



XML_SHOES_ROOT

Now OFBiz storefront knows:

"This catalog should start browsing from
ROOT"

5. Product Features

XML:

```
<ProductFeatureCategory  
productFeatureCategoryId="XML_SHOE_SIZE"/>
```

and

```
<ProductFeature
```

```
productFeatureId="XML_SIZE_12"  
productFeatureTypeId="SIZE"/>
```

Data model entities:

ProductFeatureCategory

ProductFeature

Records:

Feature Category:

ID	Description
XML_SHOE_SIZE	Shoe Size

Feature:

ID	Type	Description
XML_SIZE_12	SIZE	12
XML_SIZE_13	SIZE	13

XML_COL COL Orang
OR_ORG OR e

Concept:

Size:

12

13

Color:

Orange

6. Virtual Product

XML:

```
<Product  
productId="MS-3"  
isVirtual="Y"  
isVariant="N"/>
```

Data model:

Product

Record:

**prod isVir isVar
uctld tual iant**

MS-3 Y N

Meaning:

This is not an actual inventory item.

Think:

Nike Air Max Shoe Page

You don't buy:

Nike Air Max

You buy:

Nike Air Max

Color: Orange

Size:12

So:

MS-3

acts like parent.

7. Add Product to Categories

XML:

```
<ProductCategoryMember  
productCategoryId="XML_SHOES_MSPORTS"  
productId="MS-3"/>
```

Data model:

ProductCategoryMember

Relationship:

MSPORTS



MS-3

Also:

PROMOTION



MS-3

Now storefront navigation becomes:

Catalog



Root

↓
Men
↓
Sports
↓
MS-3

8. Selectable Features on Virtual Product

XML:

```
<ProductFeatureAppl  
productId="MS-3"  
productFeatureId="XML_SIZE_12"  
productFeatureApplTypeId="SELECTABLE_FEATU  
RE"/>
```

Entity:

ProductFeatureAppl

Meaning:

The user can choose:

Size:

[12]

[13]

Color:

[Orange]

The UI reads these selectable features.

9. Variant Products

XML:

```
<Product  
productId="MS-3-ORANGE-12"  
isVirtual="N"  
isVariant="Y"/>
```

Entity:

Product

Inserted:

Product ID	Virt ual	Vari ant
MS-3-ORA NGE-12	N	Y

MS-3-ORA N Y
NGE-13

Actual inventory products.

10. Link Virtual → Variants

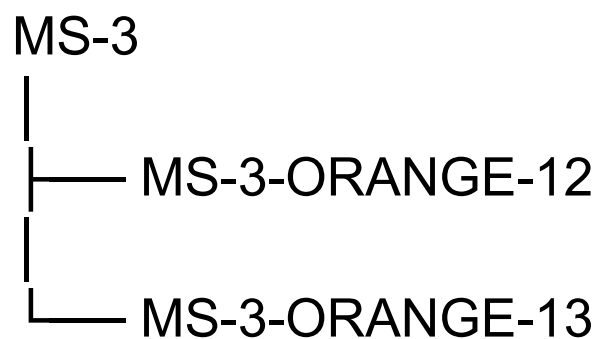
XML:

```
<ProductAssoc  
productId="MS-3"  
productIdTo="MS-3-ORANGE-12"  
productAssocTypeId="PRODUCT_VARIANT"/>
```

Data model:

ProductAssoc

Relationship:



When user chooses:

Size=12

Color=Orange

OFBiz searches:

ProductAssoc

and finds:

MS-3-ORANGE-12

11. Variant Features

XML:

```
<ProductFeatureAppl  
productId="MS-3-ORANGE-12"  
productFeatureId="XML_SIZE_12"  
productFeatureApplTypeId="STANDARD_FEATURE"  
"/>
```

Meaning:

Variant permanently owns:

MS-3-ORANGE-12



Size=12

Color=Orange

Complete flow now becomes

Catalog



Category



Virtual Product(MS-3)



Selectable Features



User chooses:

Color=Orange

Size=12



ProductAssoc lookup



Variant product found:

MS-3-ORANGE-12



Actual inventory item

12. Promotion section

XML:

```
<ProductPromo  
productPromoId="XML_SHOES_PROMO"/>
```

Data model (second image):

ProductPromo

ProductPromoRule

ProductPromoCond

ProductPromoAction

Flow:

Promotion

XML_SHOES_PROMO



Rule

Rule 01

↓

Condition

PPIP_PRODUCT_QUANT
PPC_EQ
1

Means:

If quantity ==1

↓

Action

PROMO_PROD_DISC
amount="10"

Means:

Give ₹10 discount

Category linked to promotion

XML:

```
<ProductPromoCategory  
productPromold="XML_SHOES_PROMO"  
productCategoryId="XML_PROMO_CAT"/>
```

Relationship:

PROMO CATEGORY



MS-3



PROMOTION APPLIES

Example runtime scenario

Suppose customer opens site:

Catalog



Men



Sports



MS-3

Customer selects:

Color = Orange

Size = 12

Qty=1

OFBiz internally:

Step 1

Reads:

ProductFeatureAppl

Gets options:

12,13,Orange

Step 2

Reads:

ProductAssoc

Finds:

MS-3-ORANGE-12

Step 3

Checks promotions:

ProductPromo

Condition:

Qty==1

True

Step 4

Applies:

₹10 discount

Final cart:

MS-3 Orange Size12

Price: ₹1000

Discount: ₹10

Total: ₹990